AIと書く
次の言葉を選ぶのは誰か

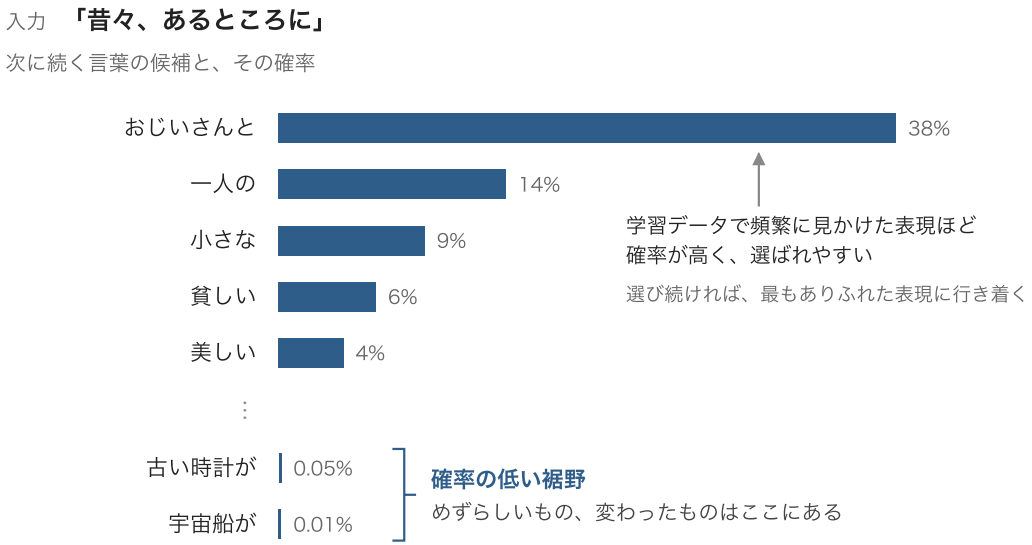
東京藝術大学芸術情報センター
田所 淳

今日の内容

- 前回の振り返り
  - 次の言葉の予測、ボタンの手前・奥・向こう、平均への引力
- ウォームアップ: 予測変換で書く
- VS Code と GitHub Copilot のセットアップ
- 補完の基本操作
- 実習: 人間の文章とAIの文章
  - AI補完なしで書く / AI補完だけで書く / どちらがAIかを当てる
- 次週までの課題
  - AIの文章を仕上げる

前回の振り返り

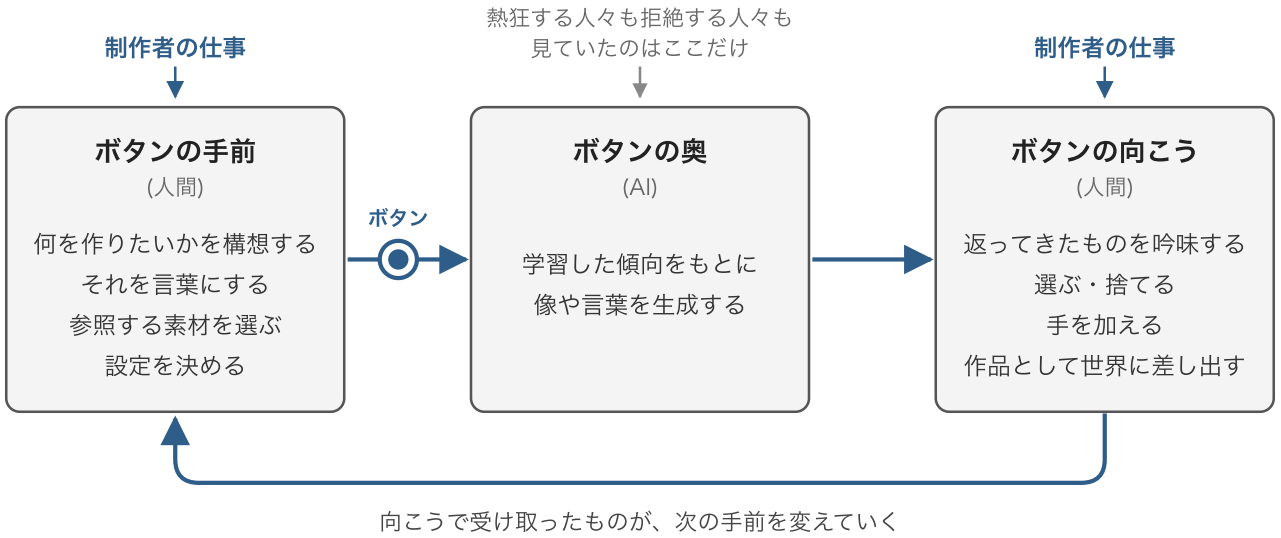
次の言葉を確率で予測する



※ 確率の値は説明のための模式的なもので、実際のモデルの出力ではありません。

- 言語モデルは、与えられた文章に続く **「次の言葉」を確率で予測** する
- 今日使う GitHub Copilot の補完も、このしくみで書きかけの文章の続きを予測する

ボタンの手前・奥・向こう



- **手前**: 構想し、言葉にし、素材を選び、設定を決める
- **奥**: AIが像や言葉を生成する
- **向こう**: 吟味し、選び、手を加え、作品として世界に差し出す

平均への引力

- 言語モデルも画像生成AIも、**確率の高い方へ** と進んで生成する
  - 「夕日」→ 水平線に沈む太陽と橙色の空という **典型的な夕日**
- めずらしいもの、変わったものは **確率の低い裾野** にある
  - そこは、AIが最も苦手とする領域
- 人間の好みに合わせる調整で、出力は **なめらかで無難な方向** へ
- 平均から抜け出すには、人間の側の粘り強い働きかけが必要
  - Allen の624回のプロンプト、九段の20万字のプロンプト

今日の目標: 「使いこなす」とは?

- 第1回の Mentimeter より
  - 「AIを使いこなせるようになりたいけれど、使いこなせるとはどういう状態なのかも分からない」
- ひとつの答え
  - **AIの提案を受け入れるか、書き換えるか、断るかを、自分で選べること**
- 今日の実習では、まず両極端を体験する
  - AIをまったく使わずに書く / AIの提案をすべて受け入れるだけで書く
- そのあいだ (選びながら書く) は、次週までの課題で

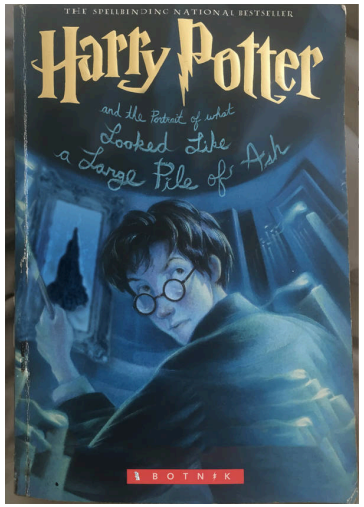
ウォームアップ
予測変換で書く

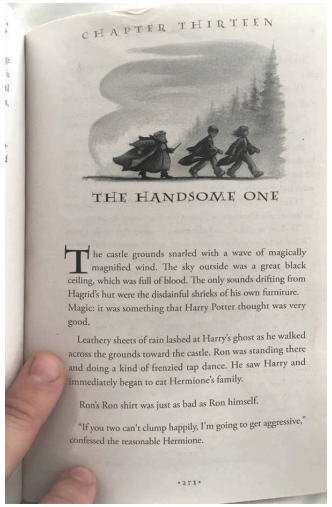
スマホの予測変換だけで書いてみよう

- スマホのメモアプリで「**生成AIによる創作とは**」と入力
- そのあとは、キーボードに出る **予測変換の候補だけ** をタップして続ける
- できあがった文章を、隣の人と見せ合う
- 予測変換も「次の言葉」を予測するしくみ
  - 入力の履歴も学習しているので、**自分の言葉の癖** が映る
  - Copilot に映るのは、膨大なテキストから学んだ **多くの人々の言葉の傾向**

> ⚠️ 見せたくない言葉が出てきたら、共有しなくてOK

Botnik Studios『ハリー・ポッター』(2017)





- 全7巻を学習させた **予測キーボード** で書かれた、架空の続編の第13章
  - メンバーが候補の言葉を一語ずつ選んで文を作り、編集者が章にまとめた
- 候補を出したのは機械、**どれを選び、どう並べるかを決めたのは人間**
  - あえて奇妙な候補を選ぶことで、作品が生まれた

画像出典: Botnik Studios「Harry Potter」

VS Code と GitHub Copilot のセットアップ

AIとの共創の準備

- 文章を書きながら、その続きを AI がリアルタイムに提案する環境をつくる
  - エディタ: **Visual Studio Code** (VS Code)
  - AI: **GitHub Copilot** の **補完機能** (書きかけの文章の続きを提案)
- 準備の流れ (前回の事前準備で1・2を済ませた人は **3から**)
  i. VS Code をインストール
  ii. GitHub アカウントを作成
  iii. VS Code で AI 機能を有効化
  iv. GitHub でサインインして認可 (Copilot Free に自動登録)
  v. Markdown で補完を有効にする
  vi. テキスト補完を試す

> 💡 **拡張機能のインストールは不要**（VS Code 1.116 以降は本体に Copilot が標準搭載）

1. VS Code をインストール

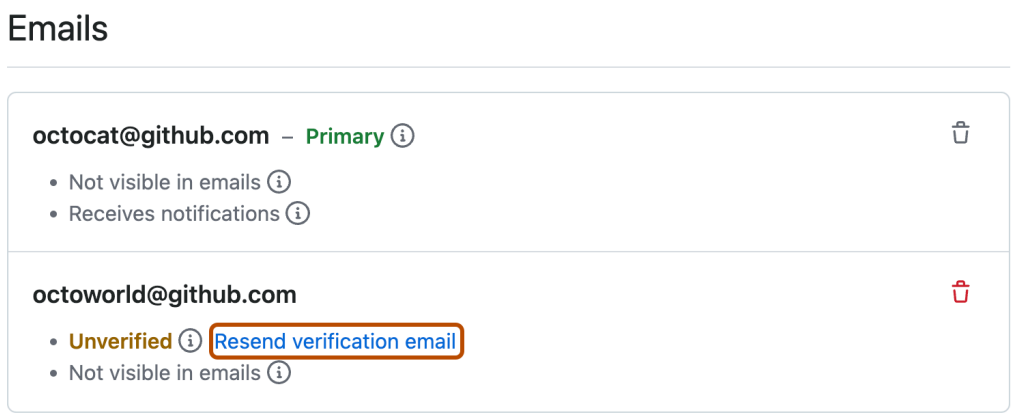
- 公式サイトからダウンロード: https://code.visualstudio.com/

OS	インストール手順
Windows	ダウンロードしたインストーラを実行（設定はデフォルトのままで OK）
macOS	zip を展開し、`Visual Studio Code.app` を「アプリケーション」フォルダへ移動

> ⚠️ **メニューの言語について**: 最初は英語で表示されます。このスライドでも英語UI名（Settings, Accounts など）で説明します。

2. GitHub アカウントを作成

- https://github.com/ を開いて **Sign up**（Google アカウントでも登録可能）
- メールアドレス・パスワード・ユーザー名・国/地域を入力し、パズルを解く
- メールに届く確認コードを入力して **メール認証を完了**
  - 認証状態は `Settings > Emails` で確認可能（Unverified なら再送できる）
- 詳細: GitHub Docs「GitHubでのアカウントの作成」

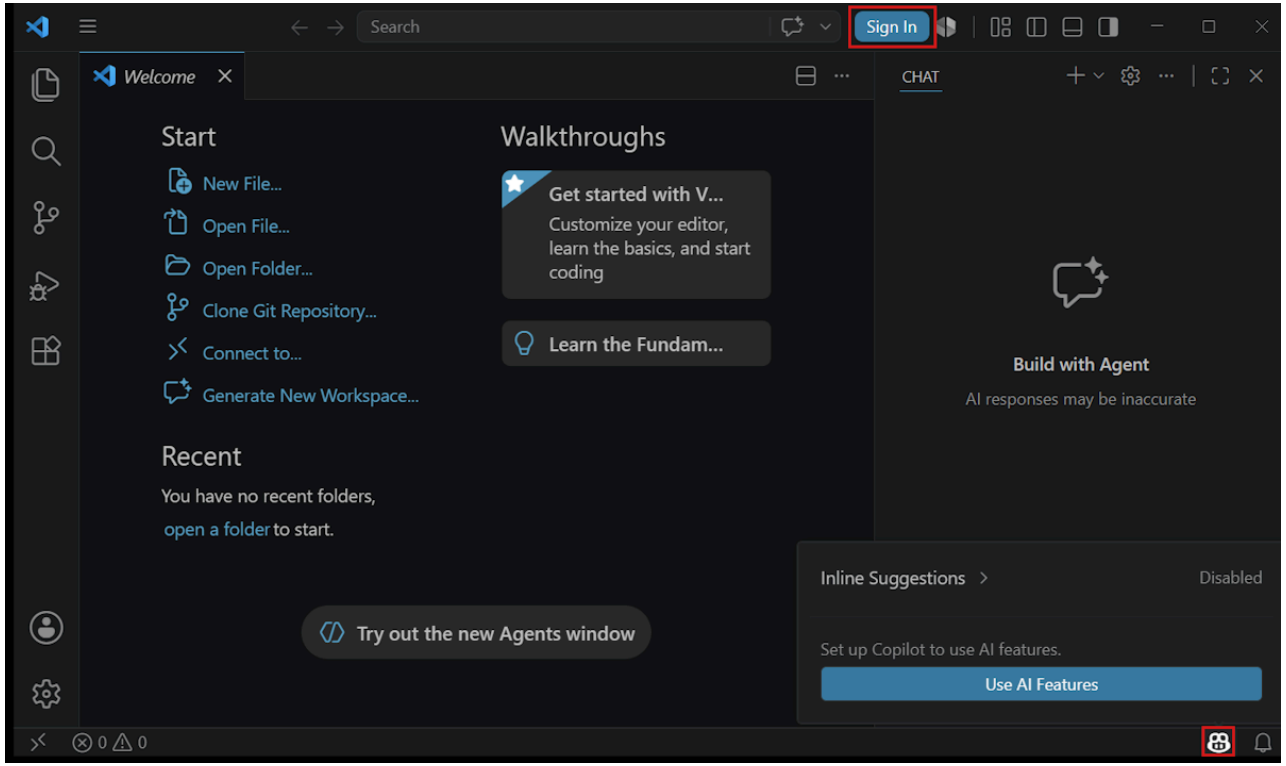


画像出典:GitHub Docs(CC BY 4.0)

3. VS Code で AI 機能を有効にする

VS Code を起動し、次のどちらかをクリック（図の赤枠参照）：

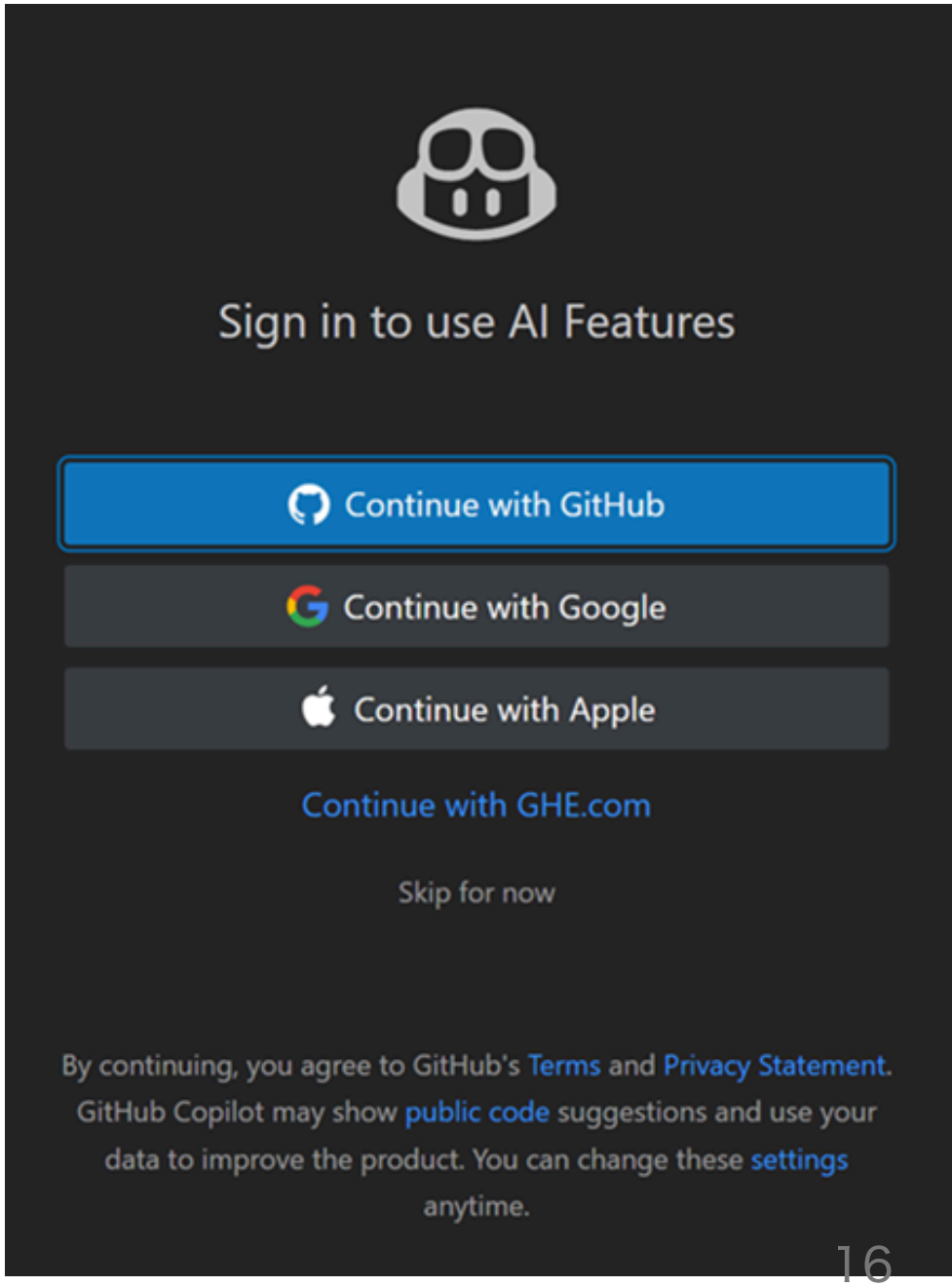
- **方法 A（推奨）**：タイトルバー右上の **Sign In**
- **方法 B**: 右下ステータスバーの Copilot アイコン → **Use AI Features**



画像出典:Visual Studio Code Docs(CC BY 3.0 US)

4-1. サインイン方法を選ぶ

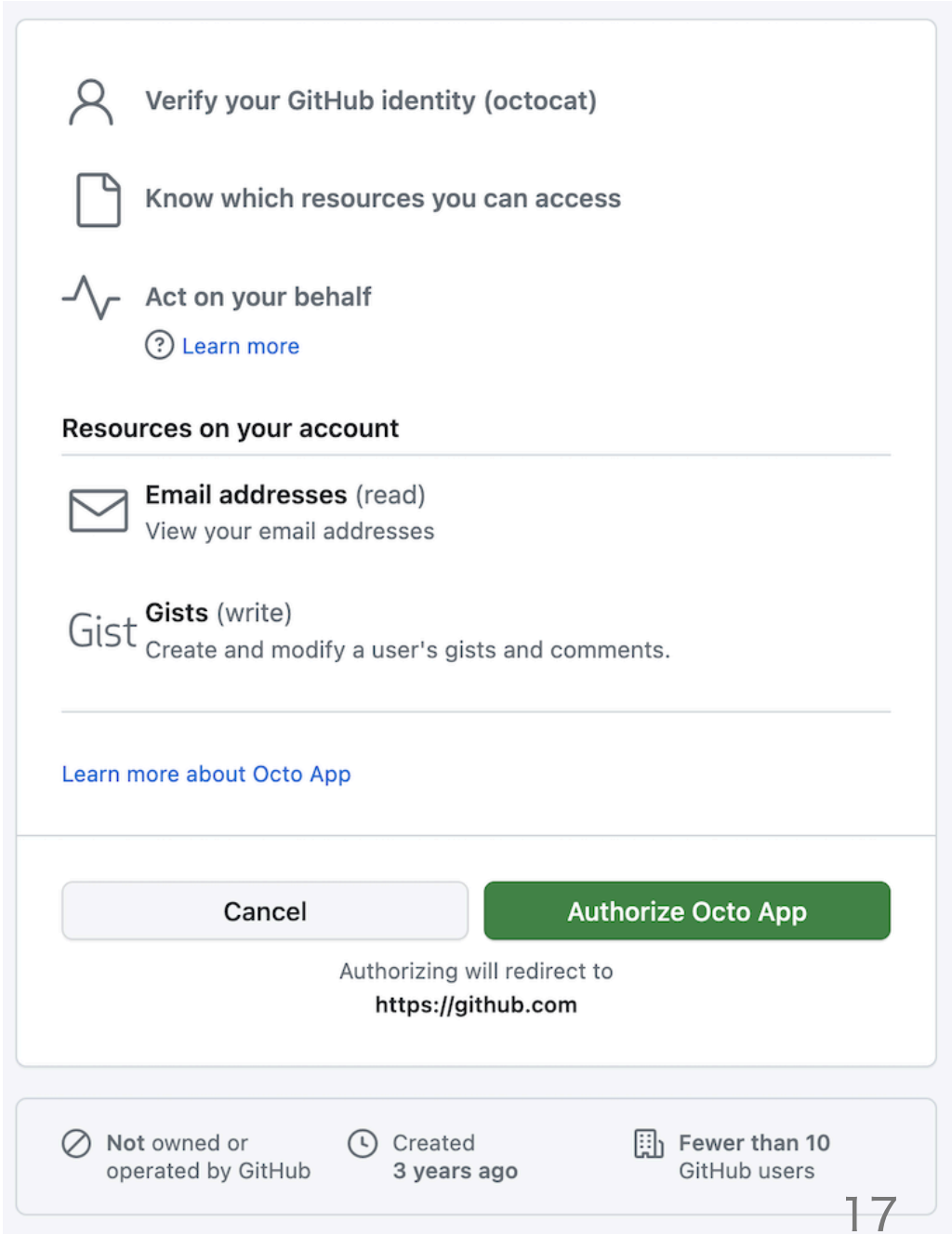
- **Continue with GitHub** を選ぶ
- 自動でブラウザが開くので、手順2で作ったアカウントでログイン
- Copilot の契約がないアカウントは、自動的に **Copilot Free**（無料プラン）に登録される
  - クレジットカードの登録などは一切不要



画像出典:Visual Studio Code Docs(CC BY 3.0 US)

4-2. ブラウザで認可する

- **認可を実行**:
  - VS Code からのアクセス許可画面が出たら、緑の **Authorize** ボタンを押す
  - （右は画面の例。実際はアプリ名が Visual Studio Code と表示されます）
- **エディタに戻る**:
  - 「Visual Studio Code を開きますか?」と聞かれたら許可して戻る
- **完了確認**:
  - 右下の Copilot アイコンを開き、プラン名（Copilot Free など）が表示されていれば完了！



画像出典:GitHub Docs(CC BY 4.0)

5. Markdown で補完を有効にする

- 今回はプログラムではなく **文章** を書くので、Markdown ファイル (`.md`) を使う
- Copilot の補完は、初期設定では **Markdown やプレーンテキストでは無効** になっている場合がある
- **有効にする方法**:
  i. `.md` ファイルを開いた状態で、右下ステータスバーの Copilot アイコンをクリック
  ii. メニューから **Markdown** での補完を有効にする
- **設定ファイルで指定する場合** (`settings.json`):

```
"github.copilot.enable": {
  "*": true,
  "markdown": true,
  "plaintext": true
}
```

データの扱いとプライバシー設定

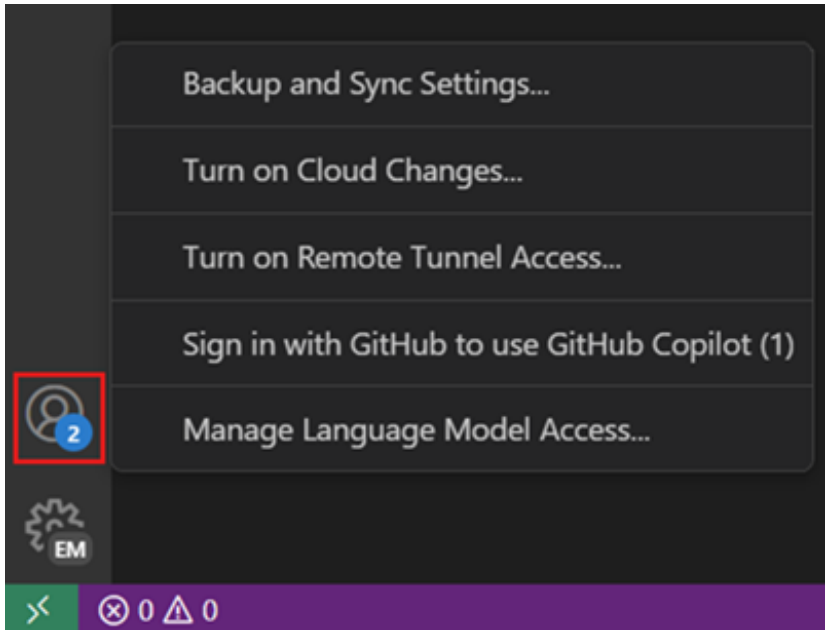
文章を書き始める前に、初期設定と変更方法を把握しておきましょう。

- **Copilot Free の初期設定**:
  - テレメトリ（製品改善のためのデータ送信）: **有効**
  - 公開コードと一致する提案（Public code suggestions）: **許可**
- **より厳格に変更したい場合**:
  - **VS Code 設定**: `telemetry.telemetryLevel` を検索して `off` に設定
  - **GitHub 設定**: 🔗 https://github.com/settings/copilot を開き、
    - 「Suggestions matching public code」を **Block** に変更
    - 「Allow GitHub to use my data for AI model training」を **Disabled** に変更

> ⚠️ 個人情報や、未発表の大切な原稿は入力しないように注意

トラブルシューティング（うまくいかないとき）

- **Sign In ボタンや Copilot アイコンが見当たらない**
  - 左下の人型アイコン（Accounts）→ **Sign in with GitHub to use GitHub Copilot**
  - VS Code のバージョンを確認（1.116 以降が必要。古ければアップデート）
- **別のアカウントでサインインしてしまった**
  - Accounts メニューから **Sign out** して正しいアカウントで入り直す
- **文章の補完が出ない**
  - ファイルを `.md` 付きで保存しているか確認
  - Markdown での補完が有効になっているか確認（手順 5）



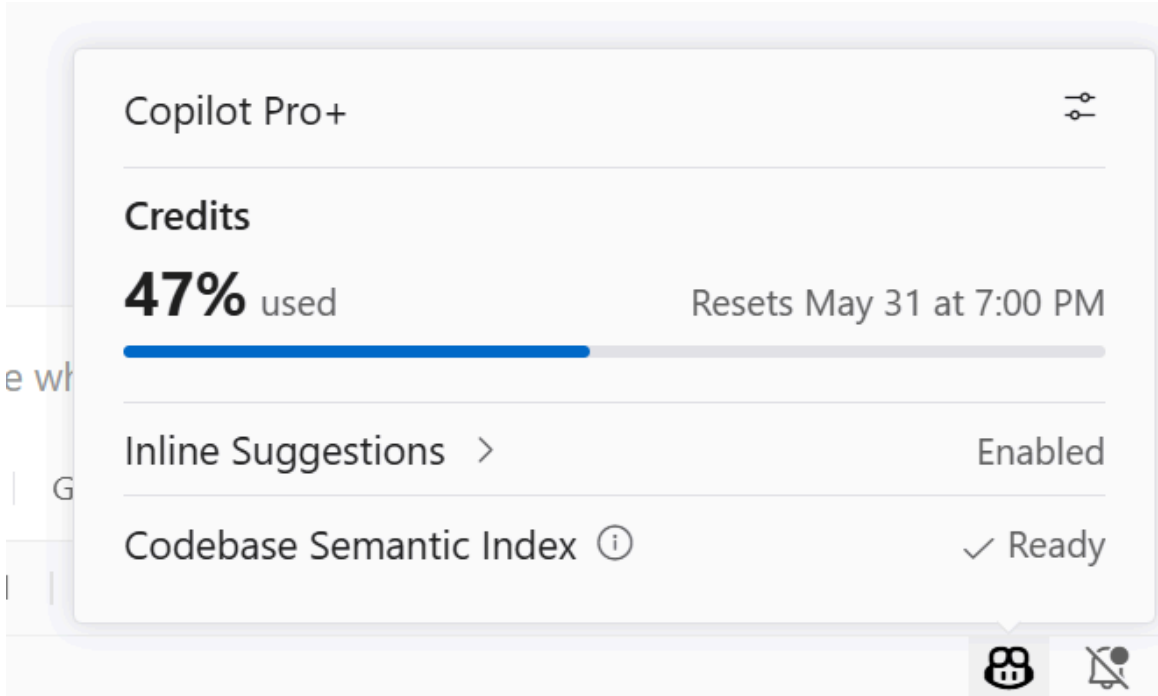
画像出典:Visual Studio Code Docs(CC BY 3.0 US)

セットアップが早く終わったら

- **使用量を確認する**
  - 今月の利用枠がどれくらい残っているかをチェック
- **GitHub Student Developer Pack を申請する**
  - 認証されれば、上限を気にせず使える (反映に数日かかることも)
- **まわりの人を手伝う**
  - 困っている人がいたら声をかけて

使用量を確認する

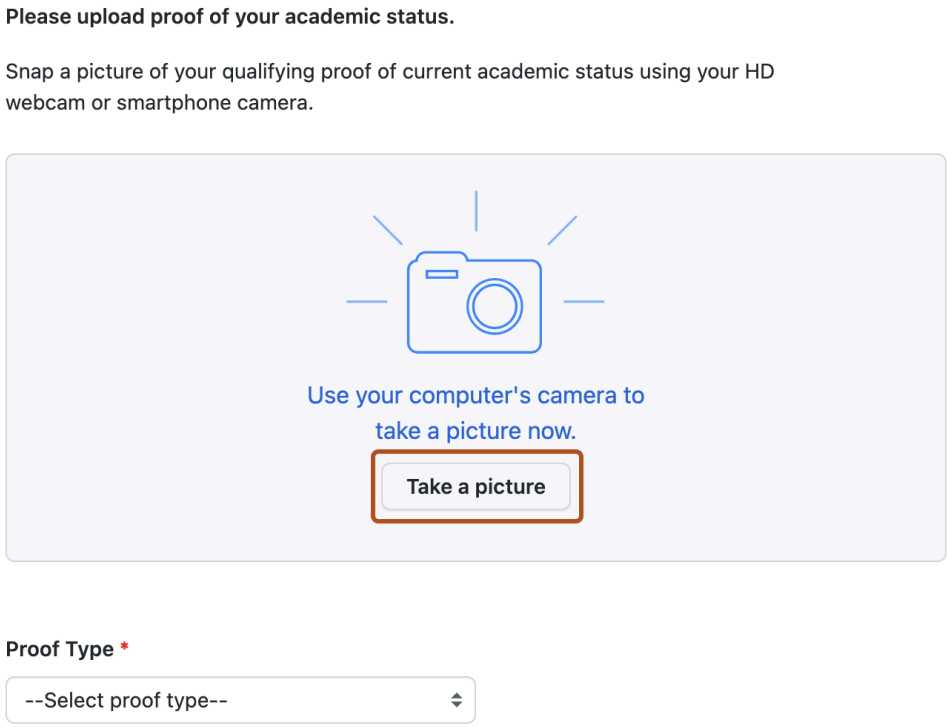
- 右下ステータスバーの Copilot アイコンをクリック
- 今月の利用枠を何%消費したかが表示される
  - （右は有料プランの例。Free ではコード補完の使用量も表示）
- **Copilot Free の月間上限**:
  - **補完**: 2,000回まで（確定ベース）
  - **チャット**: AIクレジットの範囲内（月50回程度が目安）
- 実習に入る前に、残り枠を確認しておきましょう



画像出典:Visual Studio Code Docs(CC BY 3.0 US)

上限を気にせず使うには？

- **GitHub Student Developer Pack**:
  - 学生認証を行うと、有料相当の **Copilot Student** を **無料** で利用できます
- **申請・有効化の手順**:
  i. https://github.com/settings/education/benefits にアクセス
  ii. 大学のメールアカウント (ac.jp) を登録し、学生証の写真等を提出して申請
  iii. 認証完了後、同ページで Copilot Student を有効化
  - ※ 認証反映に数日かかる場合があります
- 詳細: GitHub Docs「Copilot Student の設定」

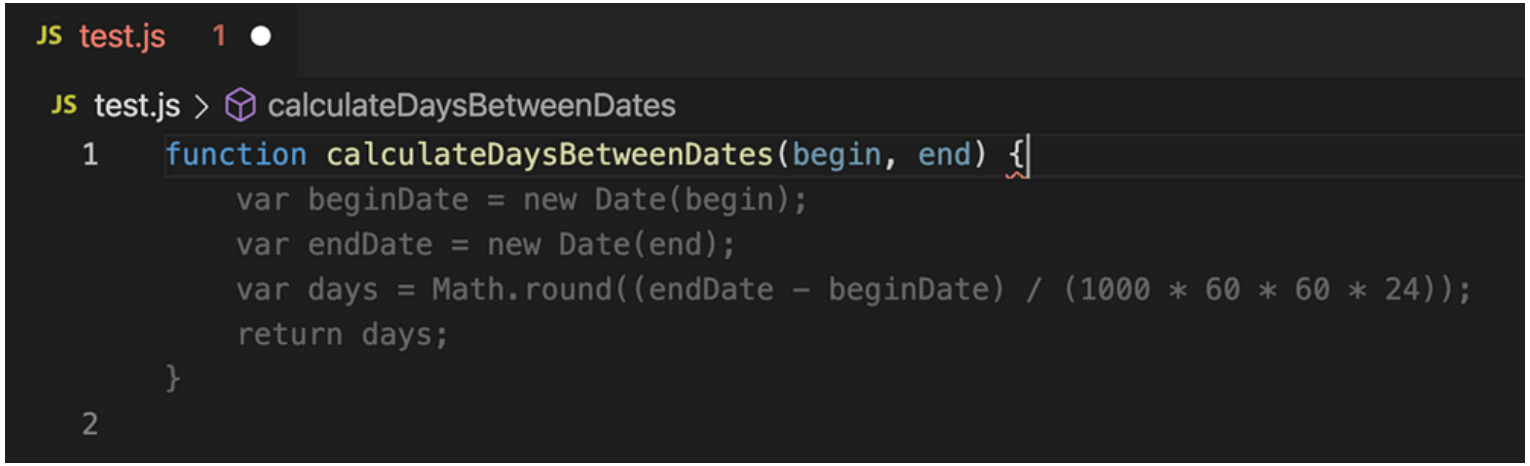


画像出典:GitHub Docs(CC BY 4.0)

補完の基本操作

6. テキスト補完を試す

- `File > New Text File` で新規ファイルを作り、`story.md` などの名前で保存
- 文章を書き始めると、続きの文章が **薄いグレー（ゴーストテキスト）** で提案される



- （図はプログラムの例。文章でも同じようにグレーの文字で続きが提案される）
- Tab: **提案を確定** / Esc: **提案を破棄**
- 👉 **試してみよう**: `# 雨の日の図書館` と見出しを書き、1行目を書き始めて少し待つ

画像出典:Visual Studio Code Docs(CC BY 3.0 US)

補完をコントロールする

- 提案を **全部は受け入れない** ことも、共創の大切な判断
- 便利な操作
  - **単語ごとに確定**: Windows `Ctrl` + `→` / macOS `⌘ Command` + `→`
    - 気に入った部分までだけ受け入れ、その先は自分で書く
  - **別の候補を見る**: Windows `Alt` + `]` / macOS `⌥ Option` + `]`
    - 提案にマウスを重ねると、候補を切り替えるツールバーも表示される
- グレーの提案は、Copilot が予測した **「確率の高い続き」**
  - 候補は一つとは限らない (前回の Transformer Explainer で見た候補のリスト)
- 提案は、直前までの文章 (= プロンプト) によって変わる
  - 書き出しの言葉、文体、見出しを変えると、続きの方向も変わる

実習
人間の文章とAIの文章

実習の流れ

- テーマ「**生成AIによる創作**」で、400字程度の文章を2つ書く
  - 書き出しはどちらも「**生成AIによる創作とは**」

	書き方	人間がすること
文章1	AI補完なしで書く	すべて
文章2	AI補完だけで書く	書き出しを入力するだけ

- 2つの文章を Google Classroom に提出 (**どちらがAIかは明かさない**)
- 提出された文章のどちらがAIかを、全員で予想して投票

文章1: AI補完なしで書く

- 右下ステータスバーの Copilot アイコン → Markdown の補完を **オフ** (手順5の逆)
- 新しいファイルを作り、`A.md` か `B.md` のどちらかの名前で保存 (どちらでもよい)
- 次の書き出しに続けて、400字程度書く

```
# 生成AIによる創作

生成AIによる創作とは
```

- ChatGPT など、ほかのAIも使わない。AIに似せたり、わざと崩したりせず、自分の言葉で
- 文字数の確認: 本文を選択すると、ステータスバーに「(120 selected)」のように表示される

文章2: AI補完だけで書く

- Markdown の補完を **オン** に戻す
- もう一方の名前 (`B.md` か `A.md`) で新しいファイルを作る
- 文章1と同じ書き出し (`# 生成AIによる創作` と `生成AIによる創作とは`) だけを入力
- あとは、提案を Tab で確定することだけを繰り返す
  - 提案が出ないときは、Enter で改行だけを入れて待つ
- 400字程度になったら止める

> ⚠️ 書き出しのあとは、文字を打つ・消す・書き換えるなどの編集を **一切しない**。別の候補 (Alt +]) や単語ごとの確定 (Ctrl + →) も使わない。同じ文が繰り返されても、そのまま残す

Google Classroom に提出する

- Google Classroom の課題に、`A.md` と `B.md` の2つのファイルを提出
- **どちらがAIの文章かは明かさない**
  - 答えは、課題の「**限定公開のコメント**」で教員にだけ伝える
  - ファイル名やクラスのコメントなど、ほかの人に見える場所には書かない

どちらがAIかを当てる

- 提出された文章を、名前を伏せてスクリーンに映す
- 組ごとに、A と B のどちらがAIの文章かを予想して **Mentimeter で投票**
- 投票のあとで答え合わせ
- 👉 考えてみよう
  - 当たった人は、何を手がかりにした? 外れた人は、どこにだまされた?
  - よく見かける言い回し、なめらかすぎる文、具体的な体験の有無、同じ表現の繰り返し…
  - 前回の **「平均への引力」** は、文章のどこに表れていた?

まとめ

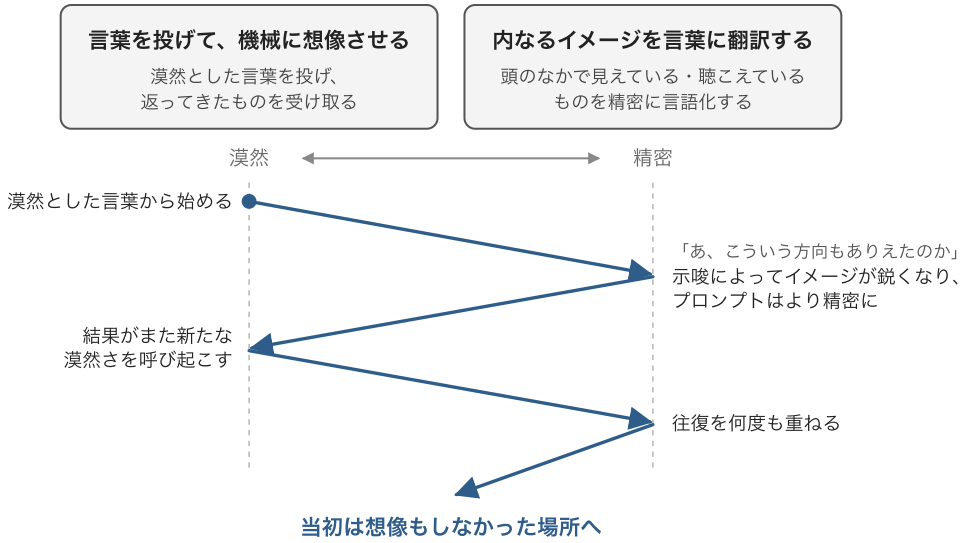
ボタンの奥だけで書かれた文章

- **文章1** (AI補完なし): 構想から言葉選び、推敲まで、すべて人間
- **文章2** (AI補完だけ): 人間が入力したのは書き出しだけ
  - 手前の仕事はほとんどなく、向こう (吟味・選択・修正) の仕事はゼロ
  - = **ボタンの奥だけで書かれた文章**
  - Botnik は候補を選んでいたが、文章2では選ぶことさえしていない
- 文章2に **向こう** を加えると、何が変わる? → 次週までの課題

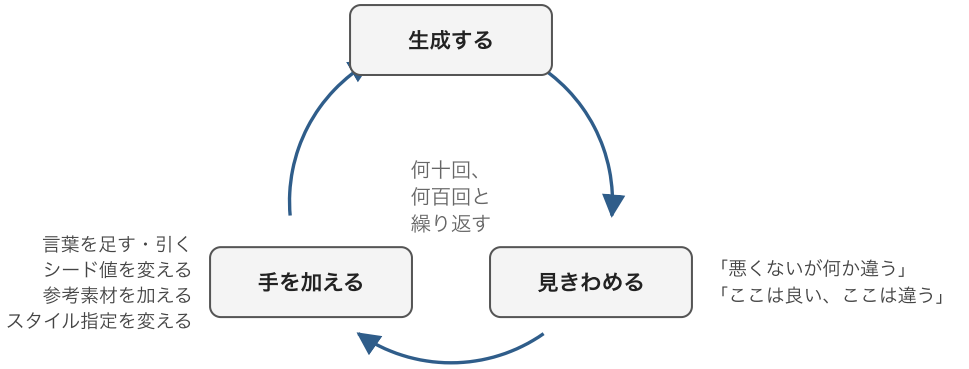
提案を選ぶ

- (b) 生成する → 見きわめる → 手を加える
  - 今日の文章2は「生成する」だけ
  - 次週の課題で「見きわめる」「手を加える」を回す
- (a) AIの返答に気づかされながら往復を重ねると、当初は想像もしなかった場所へ
- **使いこなす = 提案を受け入れる・書き換える・断るを、自分で選べること**

(a) 二つの態度と、そのあいだの往復



(b) 実践のサイクル



次週までの課題
AIの文章を仕上げる

課題: AIの文章を仕上げる

- 実習で AI補完だけで書いた文章 (文章2) を出発点に、修正を加えて仕上げる
- 修正の方法は自由 (組み合わせてもよい)
  - **自分の手で直す**: 言葉を足す・削る・書き換える・順番を入れ替える
  - **AIに修正を頼む**: 直したい部分と修正ポイントをプロンプトで指摘し、書き直してもらう
- 仕上がりは400字程度が目安
- 仕上げた文章に、**考察** を添える

AIに修正を頼むには

- 補完は「続き」を書く機能 → すでにある文章を直すには **チャット** を使う
- VS Code: 直したい部分を選択して Ctrl (⌘) + I → その場でAIに指示を出せる (インラインチャット)
- ChatGPT や Gemini など、ふだん使っているチャットAIに貼り付けて頼んでもOK
- 修正ポイントは具体的に
  - 「『可能性』や『便利』のような、よく見かける言葉を使わずに言い換えて」
  - 「3文目の主張に、反対の立場からの意見も加えて」
  - 「最後の一文を、読み手への問いかけで終わるように書き直して」

> 💡 Copilot Free のチャットは、月50回程度が目安

考察

- 仕上げた文章のあとに、400字程度の考察を書く
- 書くことの例
  - AIだけで書いた文章の、どこを、なぜ直したのか
  - 自分の手で直したところと、AIに頼んだところの違い
  - 実習の投票で、AIの文章は見破られた? その決め手は?
  - 人間が書いた文章とAIが書いた文章の違いについて、気づいたこと
  - 生成AIによる創作について、書く前と書いた後で考えが変わったところ

提出方法

```
## もとの文章（AI補完のみ）
（実習で書いた文章2を、そのまま貼る）

## 仕上げた文章
（修正して仕上げた文章）

## 考察
（400字程度）
```

- **提出物**: 上の構成で1つにまとめたテキストファイル（`.md`）
  - ファイル名は `学籍番号_氏名.md` に変更して提出
- **提出先**: Google Classroom の課題
- **締切**: 次回の授業の前日まで

参考文献

- Botnik Studios. “Harry Potter and the Portrait of What Looked Like a Large Pile of Ash.” 2017.
- Visual Studio Code Docs. “Inline suggestions from GitHub Copilot in VS Code.”
- Visual Studio Code Docs. “Inline chat and Quick Chat.”